LAB #10 — NHÓM CHUYÊN ĐỀ

PROMETHEUS & GRAFANA MONITORING

Giám sát Hạ tầng Linux Chuẩn Enterprise



Recording Rules
Metric tính sẵn, tái sử dụng



HA & LTS
Active-Active, Thanos + MinIO



Security & Auth
TLS · Basic Auth · RBAC · UFW



GitOps & MaC
Monitoring-as-Code, CI/CD



Alertmanager
Routing, Grouping, Telegram

mon-server 192.168.122.54 | mon-target 192.168.122.38 | KVM/libvirt · Debian 12 Minimal Cloud

Observability & 3 Trụ Cột

Nhìn vào bên trong hệ thống từ bên ngoài

Metrics

Hệ thống đang **khỏe hay yếu?**
CPU, RAM, Disk I/O, Request/giây

Phát hiện bất thường

Logs

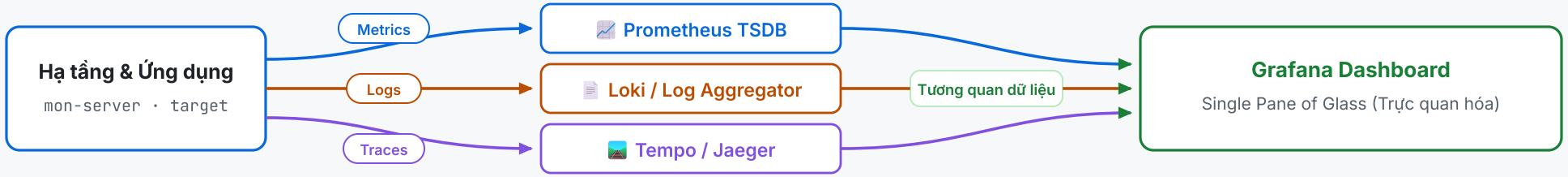
Chuyện gì đã **xảy ra** tại thời điểm đó?
Error stack, System log, Audit trail

Điều tra nguyên nhân (RCA)

Traces

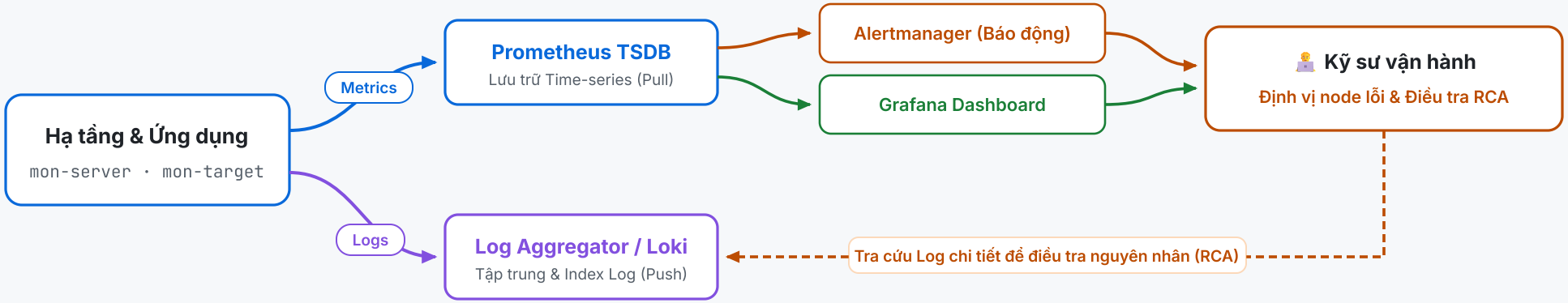
Request đi qua đâu, **chậm ở đâu?**
Span latency, Microservices call-path

Định vị điểm nghẽn



Metrics vs Logs

Tiêu chí	Metrics	Logs
Bản chất	Float64 + Labels + Timestamp	Chuỗi text / JSON
Mục đích	Phát hiện bất thường, cảnh báo	Điều tra nguyên nhân (RCA)
Dung lượng	~1–2 bytes / điểm đo	Vài chục MB đến GB / ngày
Truy vấn	Cực nhanh (PromQL)	Chậm hơn, cần full-text index



Vì Sao Cần Giám Sát?

Chủ động
Phát hiện sự cố trước khi người dùng báo cáo

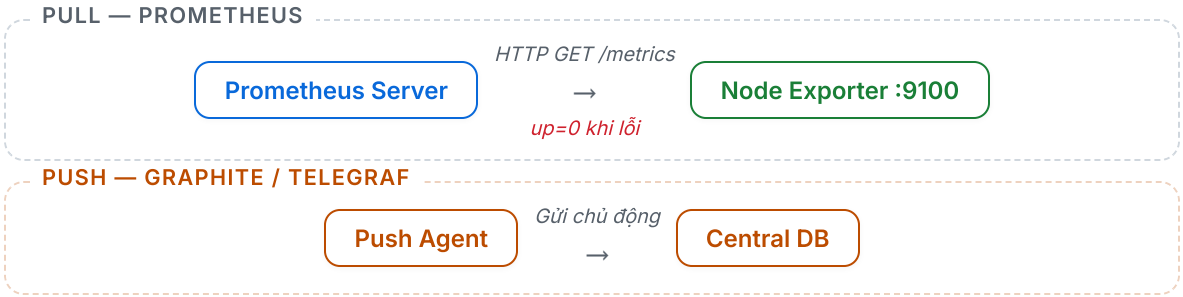
MTTD / MTTR
Giảm thời gian phát hiện và khôi phục sự cố

Capacity Planning
Dự báo tài nguyên, tránh bão hòa đột ngột

Root-Cause Analysis
Xác định nguyên nhân gốc rễ nhanh chóng



Pull vs Push



Tiêu chí	Pull	Push
Phát hiện máy sập	Scrape lỗi → up=0	Khó phân biệt
Debug	curl trực tiếp	Soi log agent
NAT / Firewall	Cần proxy	Agent mở ra ngoài
Batch job ngắn	Dùng Pushgateway	Rất phù hợp

- **CNCF Graduated** — sau Kubernetes · Pull model: server tự quyết tần suất thu thập
- Thách thức production: Alert Fatigue · HA · Long-term Storage · Security · GitOps

Bốn Loại Metric

```
# HELP node_cpu_seconds_total Seconds the CPUs spent in each mode.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 18452.73
   ── tên metric ──   ─── labels ───   ─ giá trị ─
```

Loại	Đặc điểm	Ví dụ
Counter	Chỉ tăng; reset về 0 khi restart	`node_cpu_seconds_total`
Gauge	Tăng giảm tự do	`node_memory_MemAvailable_bytes`
Histogram	Đếm theo bucket (`_bucket` , `_sum` , `_count`)	Độ trễ HTTP
Summary	Quantile tính sẵn phía client	Thời gian xử lý

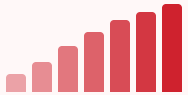
Vì Sao Counter Luôn Cần `rate()`?

$$rate(V[t]) = \Delta V / \Delta t$$

❌ SAI — Counter thô

```
node_cpu_seconds_total{mode="idle"}
```

Trả về **18452 giây** — con số khổng lồ, không có ý nghĩa tức thì

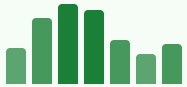


Tăng mãi

✅ ĐÚNG — Sau rate()

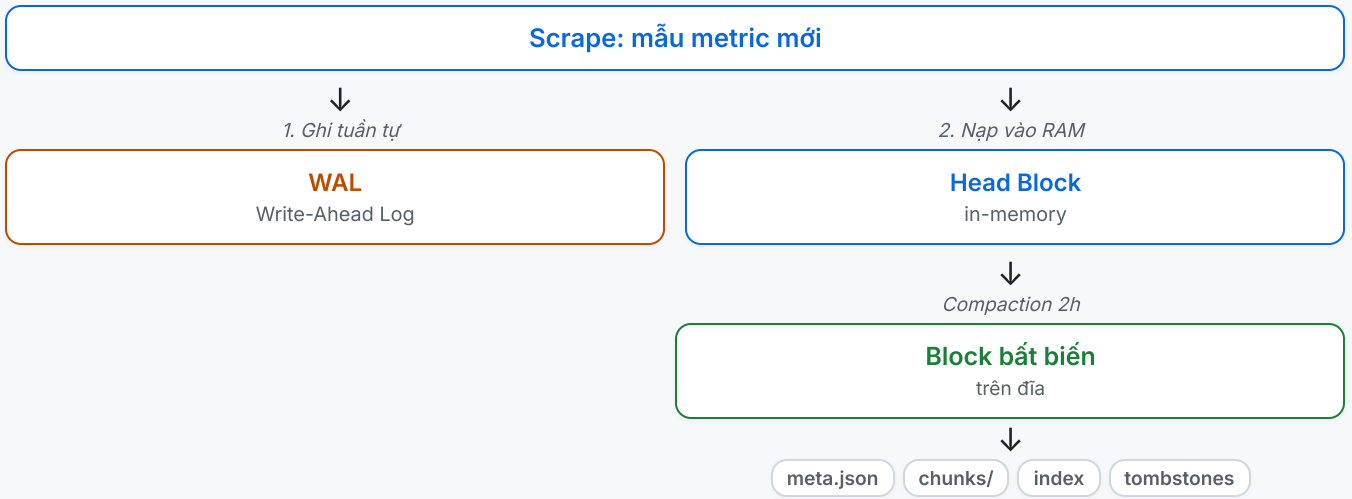
```
rate(node_cpu_seconds_total{mode="idle"}[1m])
```

Tốc độ tăng mỗi giây trong 1 phút. *rate() tự xử lý counter reset về 0.*



Dao động quanh mức ổn định

Kiến Trúc TSDB của Prometheus



~1.37

bytes / sample

Nén Gorilla (XOR delta-of-delta)

WAL — Durability

Head Block — Speed

Block — Immutable

Mục Tiêu 5 Chuyên Đề Mở Rộng

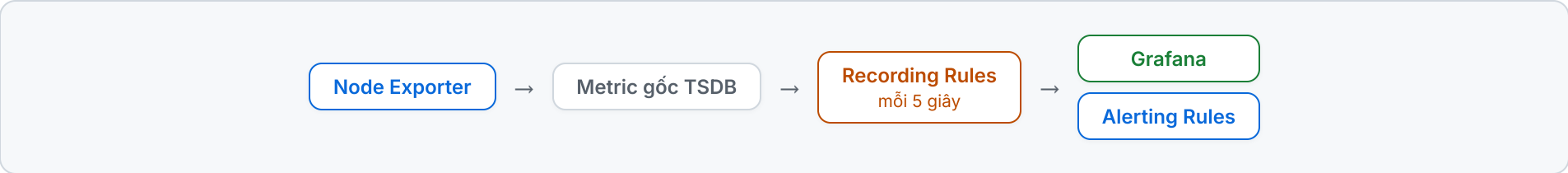
Chuyên đề	Vấn đề	Giải pháp
1. Recording Rules	Lặp công thức, khó bảo trì	Metric tính sẵn dùng chung
2. HA & Lưu trữ	Đơn điểm lỗi, giới hạn lưu trữ	HA Replicas · Thanos · MinIO
3. Security & Auth	Dữ liệu và dịch vụ thiếu bảo vệ	TLS · Xác thực · Phân quyền
4. GitOps & MaC	Cấu hình thủ công, khó kiểm soát	Cấu hình bằng mã · Kiểm tra trước
5. Alertmanager	Cảnh báo trùng lặp, khó định tuyến	Grouping · Routing · Telegram Notification

Recording Rules → HA + LTS → Security → GitOps → Alertmanager → Enterprise-Ready

PHẦN 1 / 5

CHUẨN HÓA TRUY VẤN VỚI RECORDING RULES

Tính sẵn metric · Tái sử dụng cho dashboard và cảnh báo



1.1. Vấn Đề Tồn Đọng & Phương Pháp

- **Tính toán lặp:** Dashboard và alert cùng đánh giá biểu thức CPU, RAM, đĩa
- **Khó bảo trì:** Công thức nằm nhiều vị trí — thay đổi dễ bỏ sót hoặc lệch cách tính
- **Giải pháp:** Recording Rules tích hợp sẵn trong Prometheus — không thêm dịch vụ

❌ **Trước — PromQL lặp nhiều nơi**

```
# Dashboard Panel 1:
100 - avg(rate(node_cpu_seconds_total
  {mode="idle"}[1m]))*100

# Alert Rule (lặp y chang):
100 - avg(rate(node_cpu_seconds_total
  {mode="idle"}[1m]))*100 > 80

# Panel 2 (window khác → kết quả lệch):
100 - avg(rate(node_cpu_seconds_total
  {mode="idle"}[5m]))*100
```

✅ **Sau — Recording Rule một lần**

```
# Định nghĩa 1 lần trong rule file:
record: instance:cpu_usage:percent
expr: 100 - (avg by (instance)(
  rate(node_cpu_seconds_total
    {mode="idle"}[1m]))*100)

# Dashboard & Alert đều dùng chung:
instance:cpu_usage:percent
instance:cpu_usage:percent > 80
```

1.2–1.3. Thiết Kế Cấu Hình & Tích Hợp (Grafana / Alert)

```yaml
groups:
  - name: linux_host_recording_rules
    interval: 5s
    rules:
      - record: instance:cpu_usage:percent
        expr: |
          100 - (avg by (instance) (
            rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

5 Rules (1.2)
CPU · RAM · Đĩa gốc · Mạng nhận · Mạng gửi

`record / expr`
Tên metric đầu ra / Công thức PromQL

`5s / [1m]`
Chu kỳ đánh giá / Cửa sổ tính tốc độ

Chỉ số	Metric dùng chung (record)	Tích hợp Dashboard & Alert (1.3)
CPU	`instance:cpu_usage:percent`	Grafana Dashboard & Alert CPU > 80% (for 1m)
RAM	`instance:memory_usage:percent`	Grafana Dashboard & Alert RAM > 85%
Đĩa gốc	`instance:root_disk_usage:percent`	Grafana Dashboard & Alert Disk > 90%

1.4–1.5. Nạp Cấu Hình, Demo & Giới Hạn

```
# Kiểm tra trước khi reload
promtool check rules recording_rules.yml
promtool check config prometheus.yml
sudo kill -HUP $(systemctl show \
  -p MainPID --value prometheus)

# Tạo tải CPU thử nghiệm
stress-ng --cpu $(nproc) \
  --cpu-load 100 --timeout 180s
```

Kiểm chứng	Kết quả kỳ vọng
So sánh CPU vs công thức gốc	Khớp timestamp
Grafana khi tạo tải	CPU tăng / giảm rõ
CPU > 80% liên tục 1m	Alert → Firing

✓ Giải quyết được

- Công thức tập trung một nơi
- Kết quả nhất quán dashboard ↔ alert
- Giảm tính toán lặp tại query time

⚠ Đánh đổi

- Thêm chi phí tính toán & lưu trữ
- Không tạo lịch sử trước khi bật
- Chưa có benchmark tốc độ cụ thể

PHẦN 2 / 5

TÍNH SẴN SÀNG CAO & LƯU TRỮ DÀI HẠN

2.1. Thách Thức Của Prometheus Local TSDB (SPOF · Storage · OOM)

💀

SPOF

1 node `mon-server` chết → Mù toàn bộ giám sát & tê liệt cảnh báo



Storage Exhaustion

Scrape interval 5s → Đĩa phình nhanh, khó giữ data >1 tháng



OOM Crash

Query lịch sử 6–12 tháng → nạp hàng nghìn block vào RAM → Crash

2.2. Tối Ưu Hóa Local TSDB

```
# /etc/fstab — loại bỏ access time write
LABEL=prometheus_data /var/lib/prometheus \
  ext4 defaults,noatime 0 2

# Prometheus service flags
--storage.tsdb.retention.time=15d
--storage.tsdb.retention.size=10GB
--storage.tsdb.wal-compression    # Snappy
```

- `noatime` — Triệt tiêu I/O cập nhật access time khi đọc chunk
- `retention.time=15d` — Tự động xóa block cũ sau 15 ngày
- `retention.size=10GB` — Xóa sớm nếu đĩa chạm 10 GB

30–50%

Giảm RAM & đĩa của WAL

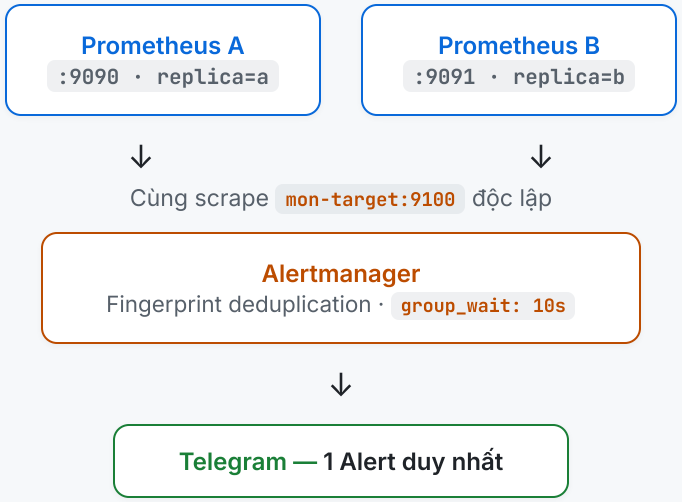
nhờ nén Snappy

Retention Kép

Xóa theo **thời gian** và **dung lượng**

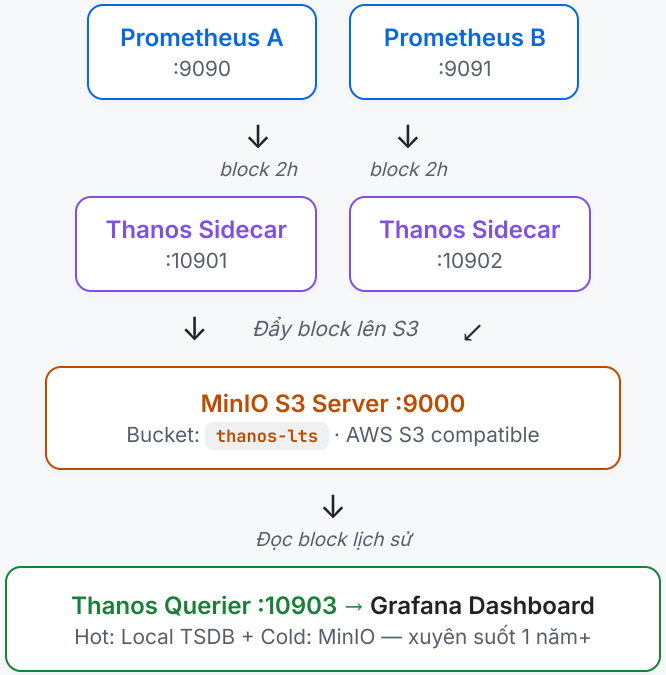
Điều kiện nào đến trước → kích hoạt trước

2.3. Kiến Trúc Dual-Instance Active-Active



- Cả 2 instance scrape **độc lập** — nếu A chết, B vẫn tiếp tục
- Alertmanager băm nhãn alert → **Fingerprint** → loại trùng lặp tự động

2.4. Lưu Trữ Dài Hạn: Thanos & MinIO



Thanos Sidecar
Gắn liền TSDB, đẩy block 2h lên MinIO

MinIO Object Storage
Tương thích AWS S3 API, triển khai gọn nhẹ

Thanos Querier
Hợp nhất dữ liệu nóng + lạnh thành 1 API

2.5. Demo & So Sánh Giải Pháp Lưu Trữ

🎬 Demo HA Failover

1. Chạy `stress-ng` → Alert `HighCPUUsage` chuyển FIRING **2.** Tắt Prometheus A: `systemctl stop prometheus` **3.** Kết quả: Alertmanager vẫn FIRING từ Prometheus B · Không gián đoạn · Không trùng lặp

Tiêu chí	Local TSDB	HA Replicas	HA + Thanos & MinIO
Mức sẵn sàng	Thấp (SPOF)	Cao (Active-Active)	Tối đa
Thời gian lưu trữ	15 ngày	15 ngày	Không giới hạn
Chi phí ổ cứng	Đắt (SSD máy chủ)	Đắt gấp đôi	Rất rẻ (Object Storage)

PHẦN 3 / 5

BẢO MẬT & QUẢN LÝ QUYỀN TRUY CẬP

3.1. Mô Hình Đe Dọa (Threat Model) Hạ Tầng Giám Sát



Information Disclosure

Node Exporter :9100 lộ toàn bộ kernel, tiến trình, mạng



Eavesdropping

Chỉ số & token truyền plaintext trên mạng nội bộ



Data Destruction

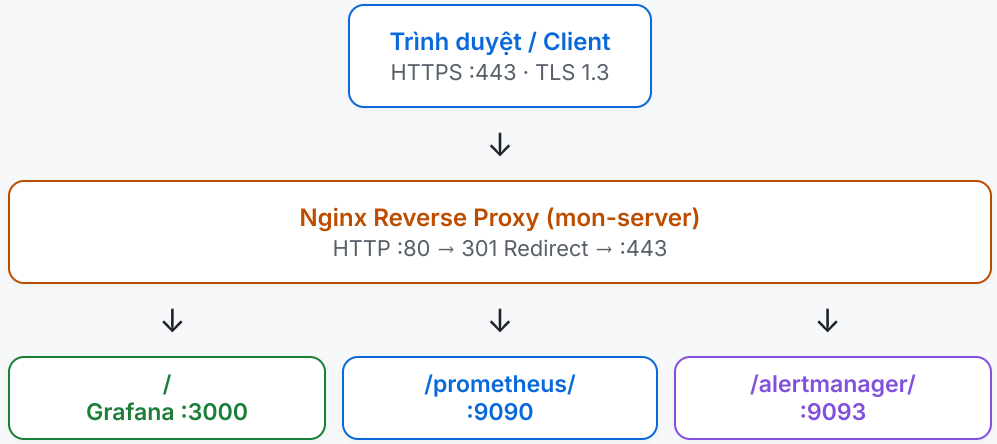
Admin API cho phép POST xóa trắng time-series



Anonymous Access

Grafana cho đăng ký tự do & xem ẩn danh

3.2. Mã Hóa Đường Truyền: Nginx TLS Reverse Proxy



- RSA 2048-bit · `TLSv1.2` + `TLSv1.3`
- HTTP → HTTPS redirect tự động (301)
- **HSTS** · `X-Content-Type-Options: nosniff`
- SSL Termination tập trung tại một điểm

Mã hóa đầu-cuối

Single Entry Point

3.3. Basic Auth (bcrypt) & Phân Quyền RBAC

```
# Tạo hash bcrypt (cost factor = 10)
htpasswd -B -C 10 web_auth.htpasswd admin

# web.yml — khai báo trong service flag
basic_auth_users:
  admin: $2y$10$...bcrypt_hash...

# prometheus.yml — scrape an toàn
basic_auth:
  username: "scraper"
  password: "ScraperStrongPassword2026!"
```

- bcrypt cost=10 — brute-force cực khó
- Cờ: `--web.config.file=/etc/prometheus/web.yml`
- `allow_sign_up = false` trong `grafana.ini`
- `[auth.anonymous] enabled = false`

👑 Admin

Quản trị datasource, user, bảo mật toàn hệ thống

🖍️ Editor

Tạo và chỉnh sửa dashboard, alert rules

👁 Viewer

Chỉ có quyền đọc — không chỉnh sửa cấu hình

3.4–3.5. UFW Firewall & Kiểm Chứng An Ninh

```
# mon-target: Cô lập Exporter
ufw default deny incoming
ufw allow 22/tcp          # SSH quản trị

# Chỉ mở :9100 cho mon-server
sudo ufw allow from 192.168.122.54 \
  to any port 9100 proto tcp

# mon-server: Bind loopback only
# :3000 :9090 :9093 → 127.0.0.1
# Mở ra ngoài: 22, 80, 443
```

- **Loại bỏ** cờ `--web.enable-admin-api` khỏi systemd
- Ngăn `POST /api/v1/admin/tsdb/delete_series`

Kịch bản Demo	Kết quả quan sát
1. Ép HTTPS	301 Moved Permanently → :443
2. Basic Auth sai	401 Unauthorized
3. Tường lửa UFW	Connection timed out



PHẦN 4 / 5

GITOPS & MONITORING AS CODE

4.1. Thách Thức Vận Hành Thủ Công (ClickOps) & Mục Tiêu GitOps

❌ ClickOps (Thủ công)

- Bấm chuột cấu hình trên Web UI
- Không có version control
- Configuration Drift
- Disaster Recovery Failure

✅ GitOps / Monitoring-as-Code

- 100% tài nguyên là code (YAML/JSON)
- Git = Single Source of Truth
- Kiểm tra cú pháp trước khi nạp
- Hot-Reload không làm rớt dịch vụ

Git Repository → Code Review → promtool check → Hot-Reload

4.2–4.3. Grafana Provisioning & File-based Service Discovery

Datasource as Code — `/etc/grafana/provisioning/`

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    url: http://localhost:9090
    isDefault: true
    editable: false  # khóa UI, chống drift
```

File SD — `prometheus.yml`

```
- job_name: "node_exporter"
  file_sd_configs:
    - files:
        - '/etc/prometheus/targets/*.json'
      refresh_interval: 5s
```

🔄 Disaster Recovery

`rm -f grafana.db` → restart → 100% Datasource & Dashboard tái sinh trong **< 5 giây**

Dynamic Target

Thêm IP vào `nodes.json` → Prometheus tự nạp sau **5 giây** — không reload, không restart

🚫 Static Config (cũ)

Thêm IP → sửa file → restart → Rủi ro cú pháp làm sập monitoring

4.4–4.5. Pipeline Triển Khai & So Sánh Vận Hành

```
# 1. Kiểm tra cú pháp TRƯỚC khi triển khai
promtool check config configs/prometheus/prometheus.yml
promtool check rules  configs/prometheus/*.yml
amtool check-config   configs/alertmanager/alertmanager.yml

# 2. Đồng bộ file cấu hình hệ thống
sudo cp configs/prometheus/*.yml /etc/prometheus/
sudo cp configs/prometheus/targets/*.json \
  /etc/prometheus/targets/

# 3. Đồng bộ Grafana Provisioning & Dashboards
sudo cp configs/grafana/json/*.json \
  /var/lib/grafana/dashboards/

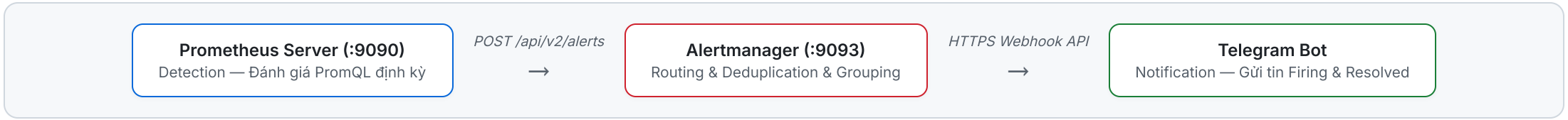
# 4. Hot-Reload — không rớt dịch vụ
curl -s -X POST http://localhost:9090/-/reload
sudo systemctl reload alertmanager
sudo systemctl restart grafana-server
```

Tiêu chí	ClickOps	GitOps
Quản lý thay đổi	Bấm chuột	Git commit + audit
Bắt lỗi cú pháp	Khi dịch vụ crash	promtool trước
Thời gian khôi phục	Mất hàng ngày	< 30 giây
Độ nhất quán	Dễ lệch cấu hình	100% đồng nhất

PHẦN 5 / 5

ĐIỀU PHỐI CẢNH BÁO VỚI ALERTMANAGER

5.1. Alert Routing · Grouping · Deduplication · Telegram Notification



Giảm Alert Fatigue

Gom hàng chục cảnh báo liên quan vào 1 tin nhắn duy nhất qua `group_wait`

Định Tuyến Đa Kênh

Phân luồng theo `severity`, `service` gửi đúng Telegram, Email, Slack

Rút Ngắn MTTR

Nội dung có ngữ cảnh chi tiết (summary, description, timestamp) giúp xử lý ngay

5.2. Cấu Hình Routing & Thông Báo Telegram

/etc/alertmanager/alertmanager.yml

```
route:
  group_by: ['alertname', 'instance', 'severity']
  group_wait: 5s
  group_interval: 10s
  repeat_interval: 2h
  receiver: 'telegram-bot'

receivers:
  - name: 'telegram-bot'
    telegram_configs:
      - bot_token: 'YOUR_BOT_TOKEN'
        chat_id: YOUR_CHAT_ID
        parse_mode: HTML
        send_resolved: true
        message: |
          {{ range .Alerts }}
          {{ if eq .Status "firing" }}🔥 CẢNH BÁO{{ else }}✅ ĐÃ KHẮC PHỤC{{ end }}
          Cảnh báo: {{ .Annotations.summary }}
          Mức độ: {{ .Labels.severity }}
          Đối tượng: {{ .Labels.instance }}
          Bắt đầu: {{ .StartsAt.Local.Format "15:04:05 02/01/2006" }}
          {{ end }}
```

Tham số	Chức năng & Ý nghĩa vận hành
`group_by`	Gom các alert có cùng nhãn vào chung một nhóm gửi
`group_wait: 5s`	Chờ gom thêm alert cùng nhóm trước khi bắn tin đầu tiên
`group_interval: 10s`	Khoảng chờ trước khi gửi bản cập nhật nhóm có alert mới
`repeat_interval: 2h`	Khoảng thời gian nhắc lại nếu alert vẫn tiếp tục FIRING
`send_resolved: true`	Tự động gửi thông báo khi sự cố đã được khắc phục

> ⚠️ **Phân biệt chu kỳ điều khiển**
>
> Các khoảng thời gian trên điều khiển **tần suất gửi thông báo** của Alertmanager. Không thay thế `scrape_interval`, `evaluation_interval` hay thời gian chờ `for` trên Prometheus.

5.3. Alerting Rules & Luồng Trạng Thái Cảnh Báo

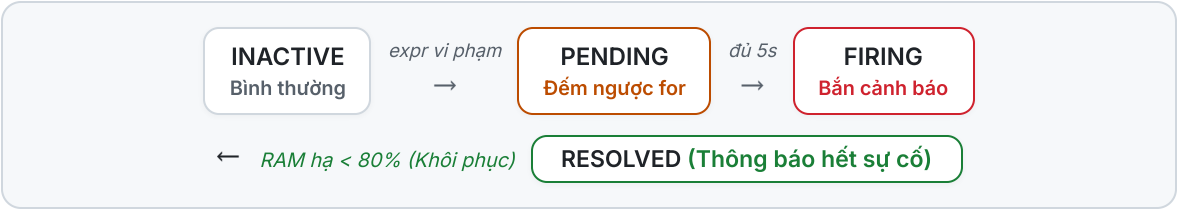
/etc/prometheus/alert_rules.yml

```
groups:
  - name: WebServerInfrastructureAlerts
    rules:
      - alert: HighMemoryUsage
        expr: (1 - node_memory_MemAvailable_bytes /
          node_memory_MemTotal_bytes) * 100 > 80
        for: 5s
        labels:
          severity: WARNING
        annotations:
          summary: "Cảnh báo dung lượng RAM!"
          description: "RAM vượt ngưỡng 80%."

# prometheus.yml – kết nối Alertmanager
alerting:
  alertmanagers:
    - static_configs:
        - targets: ['localhost:9093']
```

- `expr` — Biểu thức PromQL xác định ngưỡng lỗi
- `for: 5s` — Lọc nhiễu (chống flapping tức thời)
- `labels` — Metadata dùng để định tuyến routing
- `annotations` — Tiêu đề & mô tả hiển thị Telegram

Vòng đời chuyển trạng thái Alert (Lifecycle)



⏳ Trạng thái PENDING

Ngăn chặn cảnh báo rác khi tài nguyên chỉ tăng đột biến trong 1-2 giây rồi tự hạ.

✅ Trạng thái RESOLVED

Báo cho kỹ sư biết hệ thống đã tự phục hồi mà không cần mở dashboard kiểm tra lại.

5.4. Demo Thực Tế: Tạo Tải RAM & Khôi Phục

```
# 1. Kiểm tra tính hợp lệ cú pháp trước
promtool check rules /etc/prometheus/alert_rules.yml
promtool check config /etc/prometheus/prometheus.yml
amtool check-config /etc/alertmanager/alertmanager.yml

# 2. Cài đặt công cụ và tạo tải 85% RAM trong 180s
sudo apt install -y python3-psutil stress
python3 auto_stress.py 85 180
```

✓ Chuỗi kiểm chứng thực nghiệm

Kiểm chứng khép kín: Phát hiện lỗi (Prometheus) → Gom nhóm & lọc trùng (Alertmanager) → Gửi Telegram → Tự động báo phục hồi khi tải kết thúc.

Giai đoạn thực nghiệm	Hiện tượng & Kết quả quan sát
RAM vượt ngưỡng 80%	Alert chuyển sang **PENDING** trên UI :9090
Duy trì đủ `for: 5s`	Alert chuyển sang **FIRING**; đẩy sang :9093
Alertmanager xử lý	Telegram nhận tin 🔥 **CẢNH BÁO RAM > 80%**
Tải stress kết thúc (180s)	RAM hạ < 80% → Telegram nhận tin ✅ **ĐÃ KHẮC PHỤC**

⚠️ Giới hạn & Lưu ý an toàn

`auto_stress.py` tính dung lượng cấp phát xấp xỉ; cần theo dõi VM để tránh kích hoạt Linux Kernel **OOM Killer** tắt nhầm dịch vụ monitoring.

5.5. Xử Lý Sự Cố (Troubleshooting) & An Toàn

Hiện tượng lỗi	Điểm kiểm tra & Khắc phục
Telegram không nhận tin	Kiểm tra Bot Token, Chat ID, DNS và kết nối internet
Thiếu timestamp / nội dung	Lỗi cú pháp Go Template; kiểm tra `.StartsAt`
Alert không lên FIRING	Biểu thức PromQL chưa chạm ngưỡng, thời gian `for` quá dài
Alertmanager không start	Lỗi thụt lề YAML, cổng 9093 bị xung đột, kiểm tra journalctl
Không có tin phục hồi	Chưa bật cờ `send_resolved: true` trong config

```
# 1. Soi log hoạt động của Alertmanager
sudo journalctl -u alertmanager -n 100 --no-pager

# 2. Kiểm tra cổng lắng nghe dịch vụ
sudo ss -tulpn | grep -E '9090|9093'

# 3. Kiểm tra alert đang hoạt động qua CLI amtool
amtool alert --alertmanager.url=http://localhost:9093
```

🔒 Nguyên Tắc Bảo Mật Cảnh Báo

- **Không commit secret:** Bot Token và Chat ID phải dùng biến môi trường hoặc file riêng có `.gitignore`
- **Phân quyền file:** Gán quyền `chmod 640` cho file `alertmanager.yml`
- **Cô lập mạng:** Port 9093 chỉ bind localhost hoặc chỉ mở cho IP của `mon-server`

Tổng Kết & Bài Học Kinh Nghiệm

1 Recording Rules

Tập trung công thức, metric tính sẵn dùng chung. Ưu tiên tính nhất quán và khả năng bảo trì. Đánh đổi: thêm chi phí tính toán & không có lịch sử hồi tố.

2 HA & LTS

Active-Active Replicas triệt tiêu SPOF. Deduplication thông minh bằng Fingerprint. Thanos + MinIO S3 mở rộng lưu trữ vô hạn với chi phí thấp.

3 Security & Auth

Phòng thủ đa tầng: TLS 1.3 → Basic Auth bcrypt → RBAC Grafana → Khóa Admin API → UFW cô lập mạng. Chủ động bảo mật thay vì mặc định mở.

4 GitOps & MaC

Chuyển dịch sang Monitoring-as-Code: Grafana Provisioning + File SD + Script CI/CD. Bắt lỗi cú pháp trước khi nạp; khôi phục DR < 30 giây.

5 Alertmanager & Telegram

Điều phối cảnh báo thông minh qua Grouping, Deduplication và Routing. Thông báo Telegram HTML có ngữ cảnh, hỗ trợ tin báo phục hồi (Resolved). Giảm tải Alert Fatigue và rút ngắn MTTD/MTTR.

CẢM ƠN THẦY VÀ CÁC BẠN!

Q & A

Recording Rules | HA & LTS | Security & Auth | GitOps & MaC | Alertmanager

`mon-server: 192.168.122.54` | `mon-target: 192.168.122.38`

KVM/libvirt · Debian 12 · Prometheus · Grafana · Alertmanager · Thanos · MinIO · Nginx